

Boosting and Additive Models

Beatrice Clavarino

Statistical Methods for Machine Learning

Università degli Studi di Milano

September 2026

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.

1 Introduction

In this project I implemented and analysed AdaBoost, the boosting algorithm introduced by Freund and Schapire in 1997 [1].

The idea behind boosting is to build an accurate classifier out of very simple ones. The procedure is the following: a first classifier is trained on the training set, the examples it gets wrong are identified, and a second classifier is trained giving more importance to precisely those examples. The procedure is repeated for T rounds, producing T different classifiers. The final prediction is a vote among all of them, weighted so that the ones with fewer mistakes count more:

$$f(\mathbf{x}) = \text{sgn}(w_1 h_1(\mathbf{x}) + \dots + w_T h_T(\mathbf{x}))$$

As base classifiers I used decision stumps, of the form $h_{i,\tau}(\mathbf{x}) = \pm \text{sgn}(x_i - \tau)$: they look at a single feature, compare it with a threshold τ , and answer $+1$ or -1 .

Taken individually these classifiers are very weak. All that is required is that their weighted error ε_i be smaller than $\frac{1}{2}$, that is, that they do slightly better than random guessing; the quantity $\gamma_i = \frac{1}{2} - \varepsilon_i$ measures this edge. The theoretical guarantee is that the training error of the combination satisfies $\ell_S(f) \leq e^{-2T\gamma^2}$, where γ is a lower bound on the γ_i . If every classifier has even a small edge, and this edge does not vanish, the training error decreases exponentially in the number of rounds.

I applied the algorithm to three binary classification problems, two synthetic and one real, in order to study three aspects: how training and test error evolve as T grows, what happens inside the algorithm while it runs, and how the result compares with simpler models.

2 The algorithm

2.1 The scheme

AdaBoost operates on a training set S of m examples $(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_m, y_m)$ with labels in $\{-1, +1\}$. At each round it maintains a probability distribution P_i over the example indices: $P_i(t)$ is the importance assigned to example t in that round, and the weights sum to one.

The scheme is as follows. We initialise $P_1(t) = 1/m$, so that all examples count equally. Then, for $i = 1, \dots, T$:

1. a classifier h_i is trained on the training set weighted by P_i ;
2. its weighted error $\varepsilon_i = \sum_t P_i(t) \mathbb{1}\{h_i(\mathbf{x}_t) \neq y_t\}$ is computed;
3. if $\varepsilon_i \in \{0, \frac{1}{2}, 1\}$ the process stops (degenerate cases, discussed below);

4. the classifier is assigned the coefficient $w_i = \frac{1}{2} \ln((1 - \varepsilon_i)/\varepsilon_i)$, which determines its weight in the final decision: the lower the error, the more the vote counts;
5. the distribution is updated according to $P_{i+1}(t) \propto P_i(t) e^{-w_i L_i(t)}$, where $L_i(t) = h_i(\mathbf{x}_t) y_t$ equals $+1$ on correct predictions and -1 on wrong ones.

When $\varepsilon_i < \frac{1}{2}$ we have $w_i > 0$, so the factor equals $e^{-w_i} < 1$ on the examples that were classified correctly and $e^{w_i} > 1$ on those that were not: weight shifts towards the hard examples, and the next round, seeking the smallest weighted error, is driven to address them.

The output is $f = \text{sgn}(w_1 h_1 + \dots + w_T h_T)$, that is, the way the T classifiers jointly respond to a new example: each vote weighs as much as its coefficient w_i , and the decision is the sign of the sum.

2.2 The coefficient w_i

The coefficient w_i is not an arbitrary choice: it is obtained by minimising an upper bound on the training error, derived by bounding the zero-one loss with the exponential loss. With this choice we have

$$\ell_S(f) \leq \prod_i \sqrt{1 - 4\gamma_i^2} \leq e^{-2T\gamma^2}$$

where $\gamma_i = \frac{1}{2} - \varepsilon_i$ measures how much better than chance each classifier is, and the last step assumes $\gamma_i \geq \gamma > 0$ for every i .

2.3 What the bound does not guarantee

The bound $\ell_S(f) \leq e^{-2T\gamma^2}$ concerns the training error only. It says nothing about the test error, that is, about how the classifier will behave on unseen data. The distinction is central in what follows: in some experiments the training error keeps decreasing while the test error gets worse.

It should also be noted that the assumption $\gamma_i \geq \gamma > 0$ is a genuine hypothesis, not a consequence of the algorithm. If the available weak classifiers are too poor for the problem, the γ_i may tend to zero and the bound becomes vacuous: $e^{-2T\gamma^2}$ tends to one, that is, it promises nothing. One of the experiments displays exactly this situation.

3 Implementation

The code is organised into separate modules: `datasets.py` generates and loads the data, `stump.py` contains the weak classifier, `adaboost.py` the main loop, `baselines.py` the comparison models, while `figures.py` and `tables.py` produce respectively the figures and the numerical values reported here.

3.1 Finding the optimal stump

A stump is identified by three parameters: the feature to examine, the threshold τ , and the polarity ± 1 . The `fit` method explores the space of these combinations and returns the one with the smallest weighted error. Since the class of predictors is small, exhaustive search is feasible and guarantees the exact optimum rather than an approximation: this amounts to empirical risk minimisation over the class of stumps.

The weighted error is computed without explicit loops, exploiting boolean indexing in `numpy`: the mask `predictions != y` selects the misclassified examples and their weights are summed. This is the direct translation of $\varepsilon_i = \sum_t P_i(t) \mathbb{I}\{h_i(\mathbf{x}_t) \neq y_t\}$.

The comparison with the threshold is strict ($x_i > \tau$), consistently with the definition $\text{sgn}(z) = +1$ if $z > 0$ and -1 otherwise, and the same convention is used both when searching for the optimal threshold and when computing predictions.

3.2 Candidate thresholds

The possible thresholds are infinitely many, but only finitely many of them induce distinct partitions of the training set. If two thresholds fall between the same two consecutive values of the feature, they induce the same partition of the examples and therefore the same classifier. The partitions achievable on a feature with k distinct values are at most $k + 1$.

I therefore generated the candidate thresholds as midpoints between consecutive distinct values, rather than using the observed values themselves. On the training data the two choices are equivalent; they differ on unseen data. The midpoint maximises the distance from the observed values, preventing the decision boundary from coinciding with a training point: for the same behaviour on the observed data, it is the more robust choice on unobserved data.

3.3 Polarity is deduced, not searched for

Given a rule with weighted error ε , the same rule with inverted polarity errs exactly on the complementary examples, and therefore has error $1 - \varepsilon$. There is no need to evaluate both polarities: the error is computed for polarity $+1$ and, if it turns out to be greater than $\frac{1}{2}$, the polarity is inverted and the error replaced by its complement.

Besides halving the cost of the search, this construction has a theoretical consequence: the returned stump always satisfies $\varepsilon \leq \frac{1}{2}$. The case $\varepsilon_i > \frac{1}{2}$ foreseen by the general scheme cannot occur, and the check on degenerate cases reduces to $\varepsilon_i \in \{0, \frac{1}{2}\}$.

3.4 Degenerate cases

The scheme prescribes stopping when $\varepsilon_i \in \{0, \frac{1}{2}, 1\}$. The three cases are not, however, all reachable in the implementation. The deduction of polarity described in 3.3 takes place inside the stump search, so the returned classifier always satisfies $\varepsilon_i \leq \frac{1}{2}$: by the time the check is performed in the boosting loop, the cases $\varepsilon_i > \frac{1}{2}$ and $\varepsilon_i = 1$ have already been ruled out, and only $\varepsilon_i = \frac{1}{2}$ and $\varepsilon_i = 0$ remain to be handled.

If $\varepsilon_i = \frac{1}{2}$ the classifier does no better than chance, and inverting its polarity would bring no advantage since $1 - \frac{1}{2} = \frac{1}{2}$. Its coefficient would be zero and would not affect the final decision: it is therefore discarded and the loop stops.

If instead $\varepsilon_i = 0$ the classifier is perfect on the training set. The formula for w_i would involve a division by zero, but the classifier must be kept rather than discarded, since on its own it solves the problem. The stump is therefore stored with a conventional coefficient of 1.0 and the loop stops; being the only term in the combination, the value of the coefficient is irrelevant to the final decision, which depends only on the sign.

4 The datasets

The analysis uses three binary classification problems: two generated artificially and one real. The synthetic datasets make it possible to control exactly the structure of the problem and the amount of noise, which is impossible on real data; the real dataset verifies that the conclusions are not an artefact of the constructed ones.

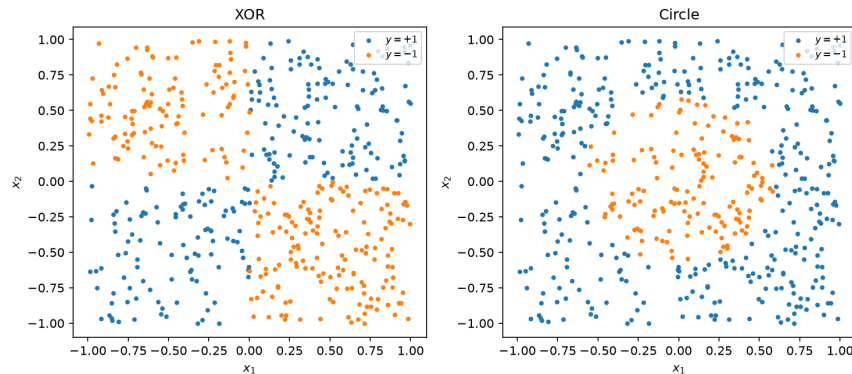


Figure 1: The two synthetic datasets. XOR on the left, the circle on the right.

4.1 XOR

The first synthetic dataset consists of m points drawn uniformly from the square $[-1, 1]^2$, with label $y = \text{sgn}(x_1 x_2)$. The label is thus $+1$ when the two coordinates have the same sign and -1 when they have opposite signs: the points form a chessboard of four alternating quadrants (Figure 1, left).

The choice is motivated by the fact that no single stump can do better than chance on this problem. A stump draws a line parallel to one of the axes; any such line leaves half the points of one class and half of the other on either side. Experimentally, the best stump under uniform weights has $\varepsilon = 0.459$.

A parameter allows a fraction of the labels to be flipped, in order to study the effect of noise.

4.2 Circle

The second synthetic dataset uses the same points in the square $[-1, 1]^2$, with label +1 if $x_1^2 + x_2^2 > 0.36$, that is, if the point falls outside the circle of radius 0.6 centred at the origin (Figure 1, right).

It was introduced as a positive counterexample: the boundary is curved, and therefore not linear, but the inequality defining it can be written as the sum of a term depending only on x_1 and one depending only on x_2 . It is thus expressible in the form that AdaBoost with stumps can represent.

A radius of 0.6 produces unbalanced classes: the area of the circle is $\pi \cdot 0.36 \approx 1.13$ against the 4 of the square, so roughly 72% of the points fall outside. The trivial classifier that always answers with the majority class therefore has error 0.272.

The experiments on the synthetic datasets use a single configuration for each family, namely one radius, one sample size and one dimensionality, and do not explore how the results vary with these parameters. The conclusions about the behaviour of the algorithm should be read within this limitation.

4.3 German Credit

The real dataset is Statlog (German Credit Data), from the UCI Machine Learning Repository [2]. It contains 1000 applicants for a bank loan, described by 20 attributes, together with the indication of whether the loan was repaid. The classes are unbalanced: 700 good payers against 300 defaulters.

The repository provides two versions: the original one, with categorical attributes encoded symbolically, and a fully numerical version prepared later by third parties. I used the former, applying one-hot encoding through `pandas.get_dummies`. The already numerical version is not accompanied by documentation of the correspondence between its columns and the original attributes: it would have been possible to say which columns the model selects, but not which attributes.

One-hot encoding avoids the artificial ordering that numbering the categories would introduce: a stump can only ask “is the value greater than τ ?”, a question devoid of meaning for arbitrarily numbered categories. The number of columns goes from 20 to 61. The original labels (1 = good payer, 2 = defaulter) were converted to +1 and -1.

5 Experimental protocol

5.1 Data splitting

Each dataset is split into a training set (70%) and a test set (30%) by a random permutation of the indices. The model is trained exclusively on the training set; the test set is used only to evaluate the final performance.

The split is not stratified: on German Credit the positive class amounts to 68.7% in the training set and 73.0% in the test set, against 70.0% overall. For this reason the reference value of the trivial classifier is computed on the actual test set (0.270) and not on the complete dataset.

5.2 Number of rounds

The number of rounds was fixed at $T = 500$ before running the experiments, uniformly across all datasets and configurations.

The choice answers a methodological requirement. In some cases the test error curves display a minimum followed by an increase, and it would be natural to select the corresponding T ; doing so, however, would mean using the test set for a decision about the model, which would thereby lose its function as an estimate of performance on unseen data. The curves are therefore reported over the whole range of rounds, and the position of the minima is commented upon as an a posteriori observation.

5.3 Reproducibility

All sources of randomness (generation of the datasets, label flipping, train/test splitting, initialisation of the comparison models) are governed by explicit seeds: every run of the scripts reproduces the values

reported below. The overfitting experiments are averaged over five seeds, since they support conclusions about the position of the minimum of the test error that a single run would not establish reliably enough; the others use a single seed, stated in each case.

5.4 Metric

All the errors reported are zero-one loss, that is, the fraction of misclassified examples. On datasets with unbalanced classes this metric must be read against the trivial classifier that always answers with the majority class: 0.272 for the circle, 0.270 on the test set of German Credit, 0.467 for XOR. An error below these values indicates that the model has extracted information from the data; an error above them indicates that ignoring the data would have been preferable.

6 Results

6.1 A representational limit: the XOR case

The first result concerns a failure, and it is useful to discuss it before the others because it clarifies how they should be read.

On XOR the training error of AdaBoost settles around 0.324 and goes no lower, however many rounds are run. The bound $e^{-2T\gamma^2}$ is not violated: its hypothesis is not satisfied, because the γ_i tend to zero, as shown in the next section.

But there is a deeper reason, independent of the algorithm. The classifier produced is $f(\mathbf{x}) = \text{sgn}(\sum_i w_i h_i(\mathbf{x}))$, and every h_i depends on a single coordinate. Grouping the terms according to the coordinate examined, the sum separates into

$$f(\mathbf{x}) = \text{sgn}(g(x_1) + h(x_2))$$

where g collects the contributions of the stumps on the first feature and h those on the second. The model is therefore additive: the decision depends on the sum of two independent contributions, not on the way the coordinates interact.

This is incompatible with XOR. Let a and a' be the values taken by g for $x_1 > 0$ and $x_1 < 0$, and b , b' those of h for $x_2 > 0$ and $x_2 < 0$. Classifying the four quadrants correctly would require $a + b > 0$, $a' + b < 0$, $a' + b' > 0$, $a + b' < 0$. Adding the first and the third gives $a + a' + b + b' > 0$; adding the second and the fourth gives $a + a' + b + b' < 0$. The same quantity would have to be both positive and negative: the system is infeasible, and at least one quadrant is necessarily misclassified.

The circle, by contrast, is compatible with the additive form: $x_1^2 + x_2^2 > 0.36$ can be rewritten as $x_1^2 + (x_2^2 - 0.36) > 0$. It is this difference, rather than the greater or lesser apparent complexity of the boundary, that determines the opposite outcomes on the two datasets.

The limitation concerns the combination of boosting with depth-one stumps, not boosting itself: the argument relies solely on the fact that every h_i depends on a single coordinate. Weak learners able to examine two of them, such as depth-two trees, would not produce an additive model, and AdaBoost would solve XOR without difficulty.

6.2 Evolution of the edge γ_i

Figure 2 shows the behaviour of $\gamma_i = \frac{1}{2} - \varepsilon_i$ over the 500 rounds, for XOR and for the circle.

In both cases γ_i decreases: by shifting weight onto the hard examples, AdaBoost makes the problem progressively harder for the subsequent rounds. What distinguishes the two datasets is the level.

Dataset	ε_1	γ_1	ε_{500}	γ_{500}	$\ell_S(f)$
XOR	0.459	0.041	0.494	0.006	0.324
Circle	0.273	0.227	0.464	0.036	0.000

Table 1: Weighted error and edge at the first and last round, together with the final training error. Training on the whole dataset, $m = 1000$, seed 0.

On the circle the initial edge is $\gamma_1 = 0.227$ and remains large enough for long enough for the bound to bite: the training error reaches zero. On XOR it starts at 0.041 and falls to 0.006, and with γ of

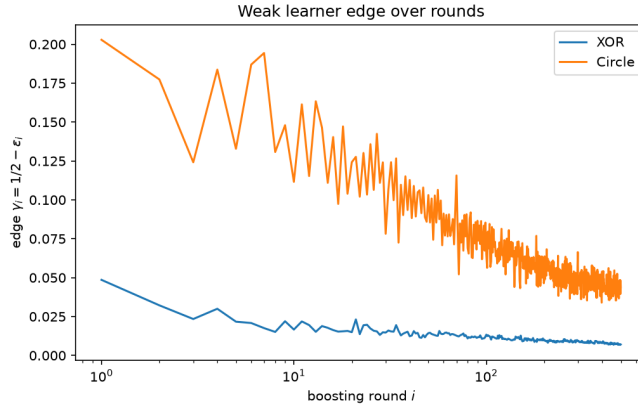


Figure 2: Edge γ_i of the weak learner over the rounds.

the order of 10^{-3} the factor $e^{-2T\gamma^2}$ is essentially one even for large T : the bound promises nothing, and indeed the error does not decrease.

The weak learnability assumption, that is, the existence of a $\gamma > 0$ bounding all the γ_i from below, is therefore not an automatic property but a condition that the problem may or may not satisfy.

6.3 Training and test error

Figure 3 reports the behaviour of the training and test error as the number of rounds grows, on three configurations: XOR, the circle without noise, and the circle with 10% of the labels flipped. The horizontal axis is on a logarithmic scale, since most of the variation occurs in the first rounds.

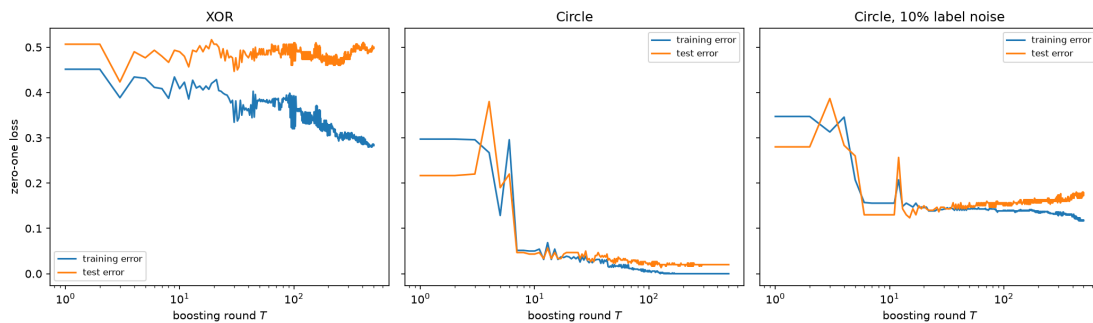


Figure 3: Training and test error over the rounds, on three configurations.

The three panels display distinct behaviours.

Circle without noise. Both curves decrease and stay close. The training error reaches zero around round 300, the test error settles at 0.022 and does not get worse over the following two hundred rounds: continuing training well beyond the point where the training error vanishes does not harm performance.

Circle with noise. The training error keeps decreasing, while the test error reaches a minimum and then rises. This is overfitting, discussed in the next section.

XOR. The training error decreases slowly to about 0.27, while the test error stays around 0.49 throughout. Over five seeds the mean values are 0.270 and 0.493, with a gap of 0.223, the widest observed in this work.

A test error of 49.3% on nearly balanced classes means that the classifier has extracted no information at all: the value is in fact slightly worse than that of the trivial classifier, which here errs on 0.467. This is not a collapse onto one class, since the model predicts +1 105 times and -1 195 times against 145 and 155 actual labels, but rather a set of decisions uncorrelated with the structure of the problem. The decrease of the training error is therefore not learning, but memorisation of particular configurations of the observed points, which does not transfer to new data.

On German Credit (Figure 4) the behaviour is analogous to that of the circle with noise, on a more contained scale: the test error reaches a minimum of 0.193 at round 94, then rises to 0.237, with a final gap of 0.058. The trivial classifier on the test set errs on 0.270, so AdaBoost improves on it by 0.033 at $T = 500$ and by 0.077 at its minimum.

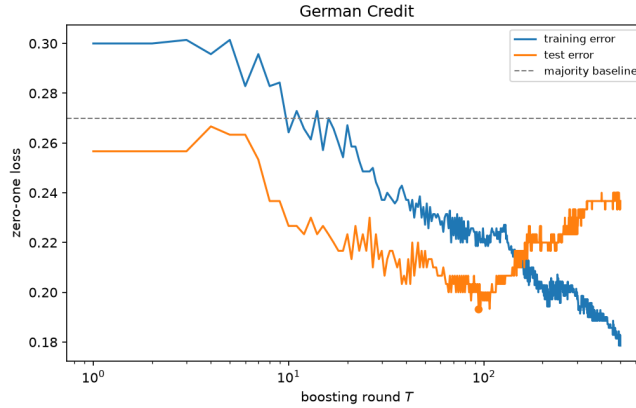


Figure 4: German Credit: training and test error over the rounds. The dashed line marks the trivial classifier.

That a real dataset should reproduce the behaviour of the synthetic one with noise, rather than the clean one, indicates that its labels are not deterministically determined by the features.

6.4 Overfitting and label noise

The synthetic dataset makes it possible to isolate the effect of noise, which cannot be done on real data where the amount of noise is unknown. The experiment repeats the same training procedure on the circle with three levels of flipped labels, 0%, 10% and 20%, averaging over five seeds.

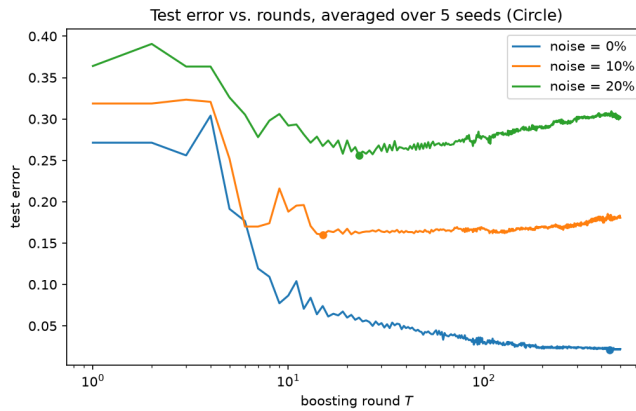


Figure 5: Test error as the noise level varies, averaged over five seeds. The markers indicate the position of the minimum.

Noise	Minimum	Round of minimum	Test at $T=500$	Train-test gap
0%	0.021	440	0.022	0.022
10%	0.160	15	0.181	0.068
20%	0.256	23	0.302	0.104

Table 2: Position of the minimum of the test error and final gap, as the label noise varies.

Without noise the minimum falls almost at the end of the range: all 500 rounds turn out to be useful. With noise it moves to round 15–23, that is, within 5% of the budget, and more than 95% of the rounds performed make matters worse rather than better.

The gap between training and test error at the end of training grows monotonically with the noise, from 0.022 to 0.104, and is the most stable measure of the phenomenon because it does not depend on where the minimum falls.

The overfitting of AdaBoost is therefore not caused by the number of rounds as such, but by its interaction with label noise: on clean data five hundred rounds do not harm a model that has already driven the training error to zero, whereas with 10% of the labels flipped some fifteen rounds are enough before adding further weak learners becomes counterproductive.

6.5 How the weights concentrate

The implementation records the distribution P_i at every round, which makes it possible to observe the mechanism that produces the overfitting. Table 3 reports, for the circle with 10% noise, the maximum weight, the mass concentrated on the ten heaviest examples, and the number of examples with weight above the uniform one.

Round	$\max_t P_i(t)$	Mass on the top 10	Examples with $P_i(t) > 1/m$
1	0.0014	0.014	0
50	0.0126	0.092	214
500	0.0114	0.085	221

Table 3: Concentration of the weight distribution, circle with 10% noise, $m = 700$.

Two phases can be distinguished. Up to round 50 the distribution concentrates rapidly: the maximum weight grows ninefold, the mass on the ten heaviest examples goes from 1.4% to 9.2%, and the number of examples deemed hard settles around 214. After round 50 the values remain substantially constant until the end.

Round 50 is approximately the point at which the test error stops improving. From there on AdaBoost keeps adding weak learners, but merely redistributes weight among the same 220 examples or so without identifying any new structure.

Since the noise was introduced artificially, it is possible to identify exactly which labels were flipped and compare them with the ranking of the weights. At round 500, 47 of the 50 heaviest examples carry a flipped label, that is 94%, against the 11.9% that would be expected in the absence of any relation (83 corrupted examples out of 700): a concentration eight times higher than chance. More than half of all the flipped labels are found in the first fifty positions.

This closes the argument about overfitting. AdaBoost increases the weight of the examples it gets wrong; an example with a flipped label contradicts the points surrounding it and is therefore impossible to classify correctly; its weight grows until it dominates the distribution. From round 50 onwards the algorithm devotes almost all of its effort to chasing examples that carry no information, and the rules thus learnt describe labelling errors rather than the structure of the problem.

6.6 Which features are selected

On German Credit the column names are interpretable, which makes it possible to examine which attributes the weak learners use. Table 4 reports the most frequently selected features over the 500 rounds, together with the total accumulated weight.

Feature	Selections	Total weight	Type
<code>credit_amount</code>	326	9.214	continuous
<code>age</code>	35	1.218	continuous
<code>duration</code>	30	0.935	continuous
<code>other_debtors_A103</code>	10	0.370	indicator
<code>purpose_A410</code>	8	0.210	indicator

Table 4: Features most frequently selected by the weak learners on German Credit, $T = 500$, seed 0.

Altogether 46 distinct features out of 61 are used: the model does not concentrate on a small subset. The distribution is nonetheless strongly asymmetric, with the loan amount selected in 326 rounds out of 500 and the three continuous features jointly accounting for 78% of the selections.

This concentration calls for caution in interpretation, because the number of candidate thresholds available for a feature depends on how many distinct values it takes: `credit_amount` has several hundred of them, an indicator column only one. Selecting the best among seven hundred thresholds amounts to choosing the predictor with smallest error within a much larger class than the one available to a binary

feature, and the training error of the winner systematically underestimates its risk: this is the same relation between the size of the class and the reliability of the estimate that governs ERM. The selection frequency therefore reflects two quantities at once, how informative a feature is and how flexible it is, and flexibility depends on cardinality, that is, on a fact about the encoding of the data. It is a direct consequence of the implementation choice discussed in Section 3.2.

The decay of the coefficients is pronounced: the first weak learner receives $w_1 = 0.424$, the last $w_{500} = 0.020$, a ratio of 21 to 1. The contribution to the final decision is therefore exhausted well before training ends, consistently with the minimum of the test error observed around round 94.

6.7 Comparison with the reference models

AdaBoost was compared with a single decision tree and with logistic regression, both in the default scikit-learn configuration and without hyperparameter search. The tree was grown without any depth limit; logistic regression is preceded by a standardisation of the features, computed on the training set alone so that no information from the test set influences the model.

The absence of tuning should be declared as a limitation: a tree with a bounded depth would presumably obtain better results on German Credit. The comparison should be read as one between default configurations rather than between tuned models; tuning the baselines without tuning AdaBoost would unbalance it in the opposite direction.

Dataset	AdaBoost	Single tree	Logistic regr.	Trivial classifier
XOR	0.500	0.003	0.407	0.467
Circle	0.020	0.030	0.217	0.272
German Credit	0.237	0.290	0.207	0.270

Table 5: Test error of the three models on the three datasets. Split 70/30, seed 0, $T = 500$.

No method prevails on all the datasets, and each of them has at least one case in which it fails.

On XOR the single tree outperforms AdaBoost by two orders of magnitude, 0.003 against 0.500. The tree combines its tests by nesting, splitting first on x_1 and then on x_2 within each branch, and this structure captures the interaction between the two features; AdaBoost combines the same tests by summation, and a sum cannot express interactions. Logistic regression, which draws a single hyperplane, fails for an analogous reason.

On the circle the situation is reversed: AdaBoost obtains the best result (0.020), the tree is close (0.030), while logistic regression only reaches 0.217 against a trivial classifier at 0.272, because no straight line approximates a circular boundary.

On German Credit logistic regression wins (0.207), followed by AdaBoost at the end of training (0.237); at its minimum, however, AdaBoost reaches 0.193. The single tree, at 0.290, does worse than the trivial classifier: grown without limits, it memorises the training set without generalising.

All the failures observed are approximation error, not estimation error: the class of representable functions does not contain a good solution, and neither more data nor more rounds would fix the problem. Logistic regression represents hyperplanes and fails where the boundary is not linear; AdaBoost with stumps represents additive functions and fails where interactions between features are needed.

Conclusion

The analysis shows that the behaviour of AdaBoost depends on the correspondence between the class of representable functions and the structure of the problem, and that overfitting is governed by label noise rather than by the number of rounds.

References

- [1] Y. Freund, R. E. Schapire. *A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting*. Journal of Computer and System Sciences, 55(1):119–139, 1997.
- [2] H. Hofmann. *Statlog (German Credit Data)*. UCI Machine Learning Repository, 1994. <https://archive.ics.uci.edu/dataset/144/statlog+german+credit+data>